

# 第51讲 实验设计

RNN 与 Transformer：路径长度、并行结构、Causal Mask 与  $T^2$  代价

## 一、实验目标

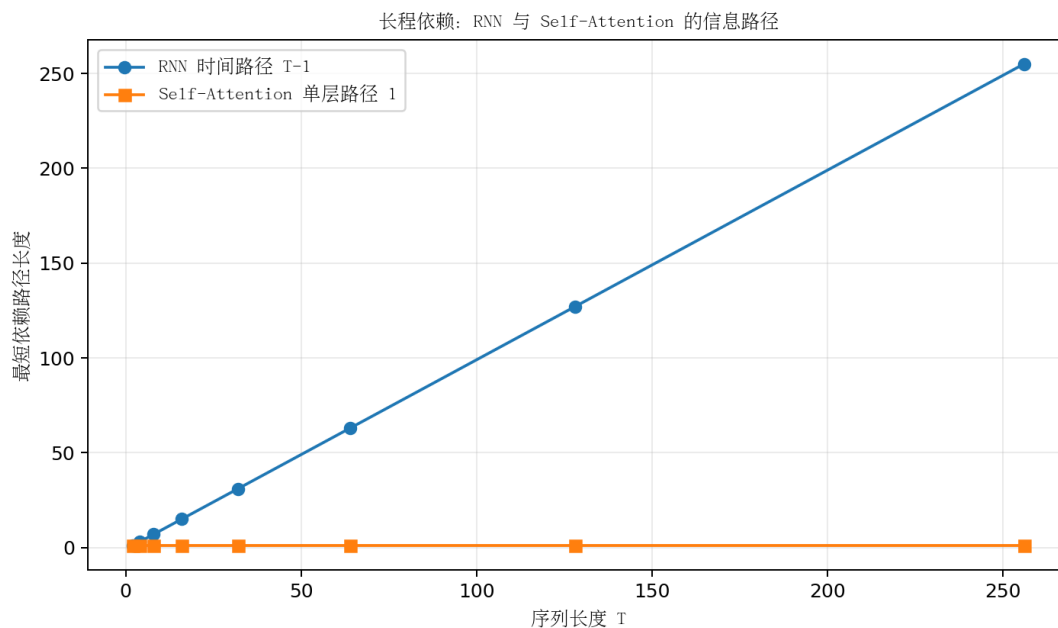
- 把“RNN 长程路径随距离增长”变成可视化曲线。
- 构造最小 Self-Attention，验证远距离 Token 可以在单层直接影响当前输出。
- 验证 Causal Mask 能禁止看未来，同时保留矩阵并行计算形式。
- 用粗略 CPU 实验展示时间递推循环与一次矩阵投影的结构性差异。
- 计算标准 Attention 的  $T^2$  score 增长，理解长上下文的代价。
- 明确训练并行与自回归生成串行的边界。

## 二、核心研究问题与假设

| RQ                      | 可验证假设                                    |
|-------------------------|------------------------------------------|
| RQ1 长程信息路径怎样变化？         | RNN 路径约 $T-1$ ；单层 Self-Attention 允许直接路径。 |
| RQ2 因果约束是否破坏训练并行？       | 不会；Mask 可以直接作用于完整 score matrix。          |
| RQ3 Transformer 的代价在哪里？ | 标准 Attention score 数量随 $T^2$ 增长。         |
| RQ4 生成是否同样并行？           | 不会；新 Token 未知导致自回归依赖。                    |

### 三、实验 1：依赖路径长度

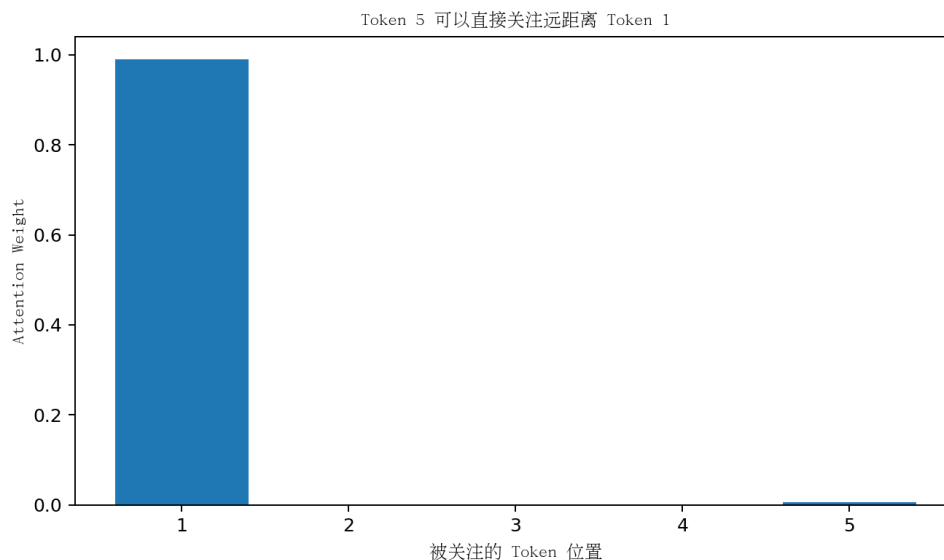
自变量：Sequence Length  $T$ 。因变量：Token 1 到 Token  $T$  的最短结构路径。RNN 使用  $T-1$ ；Self-Attention 单层记为 1。这里是架构图上的路径比较，不是实际 wall-clock。



预期：RNN 随  $T$  线性增加，Attention 保持常数级单层直接交互。

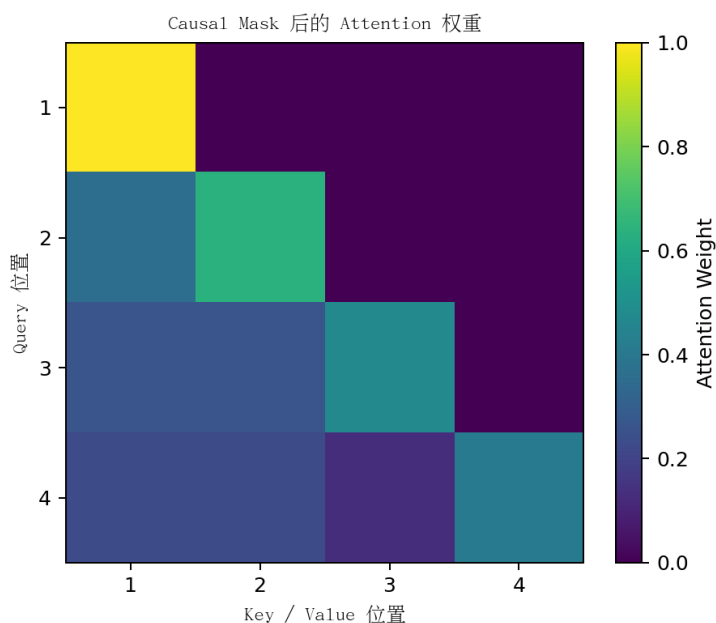
### 四、实验 2：远距离 Attention 聚合

构造 5 个 Value，故意让 Token 1 携带明显信号；人为设置 Token 5 的 score，使 Token 1 获得最高权重。观察 Token 5 输出中是否直接出现 Token 1 的 Value。



## 五、实验 3: Causal Mask

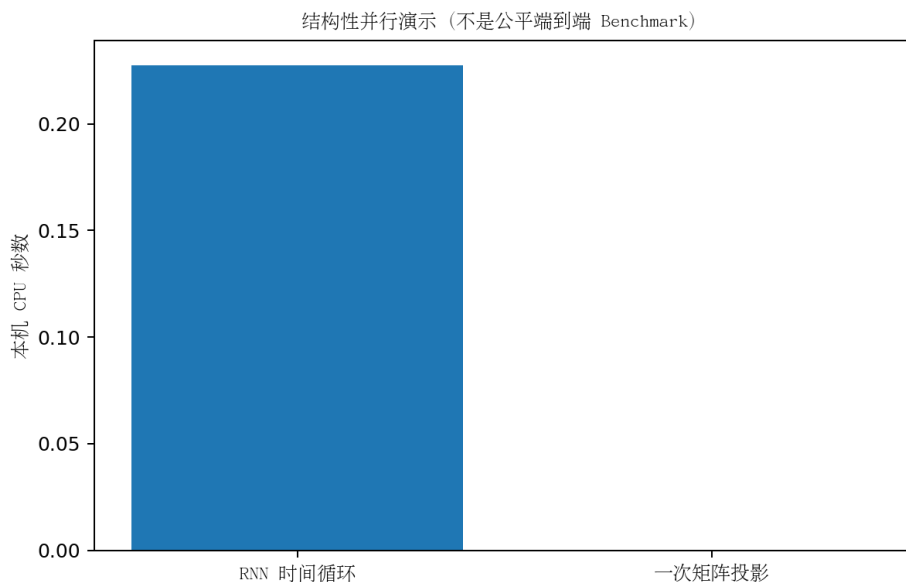
构造 4 个 Token, 令  $Q=K=V=X$ 。一次计算完整 score matrix, 然后将  $j>i$  的未来位置填成  $-\infty$ 。Softmax 后检查未来权重是否为 0。



关键判据: 未来权重为 0, 同时输出 Shape 一次得到全部 4 个位置。

## 六、实验 4: 结构性并行粗略演示

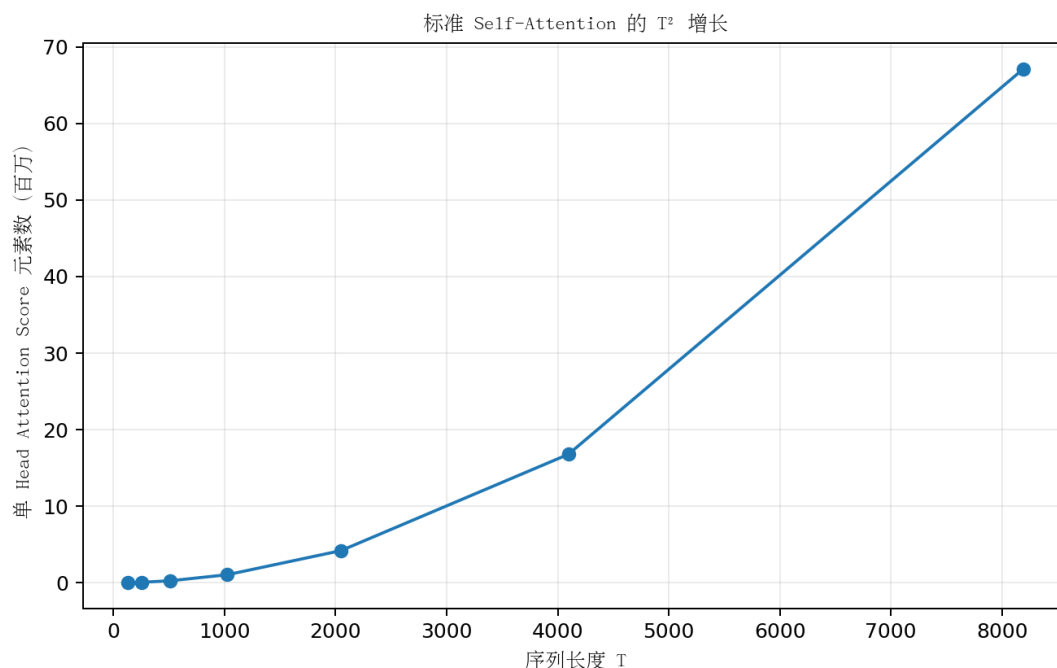
RNN 用 Python 时间循环实现  $h_t = \tanh(x_t W_x + h_{t-1} W_h)$ ; Transformer 只比较一次所有 Token 的  $Q = XW_Q$  投影。计时结果只说明“是否存在时间递推依赖”, 不能作为端到端公平 benchmark。



必须报告限制: 实现不对称、没有 fused RNN kernel、没有完整 Attention, 因此禁止用倍数直接下性能结论。

## 七、实验 5: $T^2$ Score Scaling

对  $T=128\dots8192$  计算  $T^2$ ，并按 fp16 每元素2字节估计单 Head 原始 score matrix 大小。这里只估计 score 本身，不代表完整训练显存。



## 八、实验 6: 训练并行 vs 生成串行

用文字过程展示生成第  $t$  个新 Token 时必须已有前  $t-1$  个生成结果。该实验不需要性能数值，它验证的是因果依赖结构。

## 九、实验控制与科研边界

- 固定随机种子；保存实际运行输出。
- 路径长度属于结构指标，不等于准确率。
- CPU 计时只做演示，不作为模型速度结论。
- $T^2$  score 大小只是 Attention 内存的一部分。
- 如果正式论文比较 RNN/Transformer，必须控制参数量、训练预算、数据、硬件、实现优化和多 seed。

## 十、验收标准

- 能解释为什么 LSTM 改善记忆但不消除时间串行。
- 能解释 Mask 为什么不妨碍训练矩阵化。
- 能从  $(T,d) @ (d,T)$  推出  $(T,T)$  和  $T^2$ 。
- 能区分训练并行和自回归生成。